

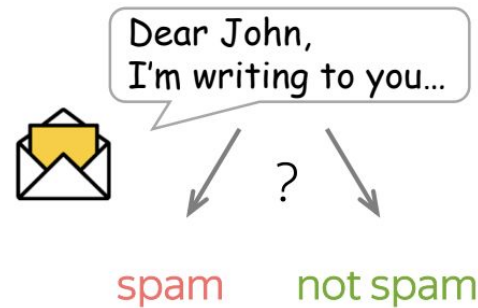
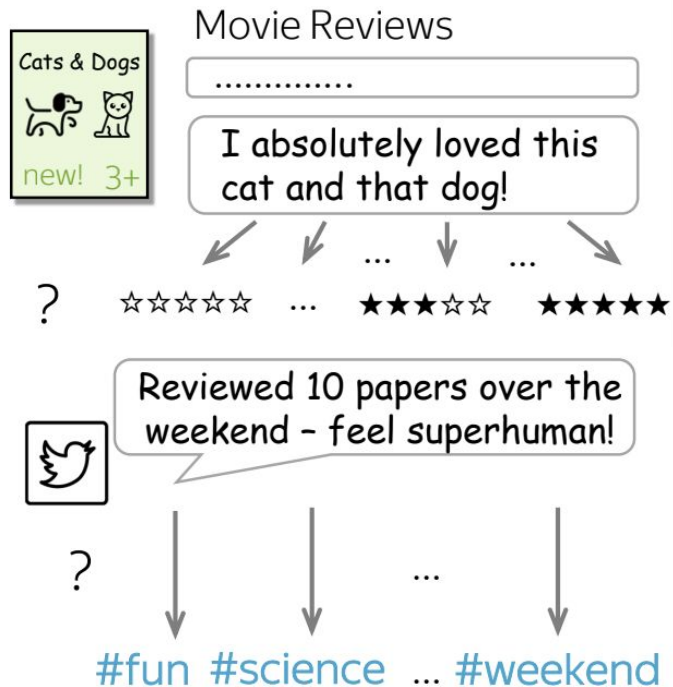
Deep text classification

CNN, RNN

План

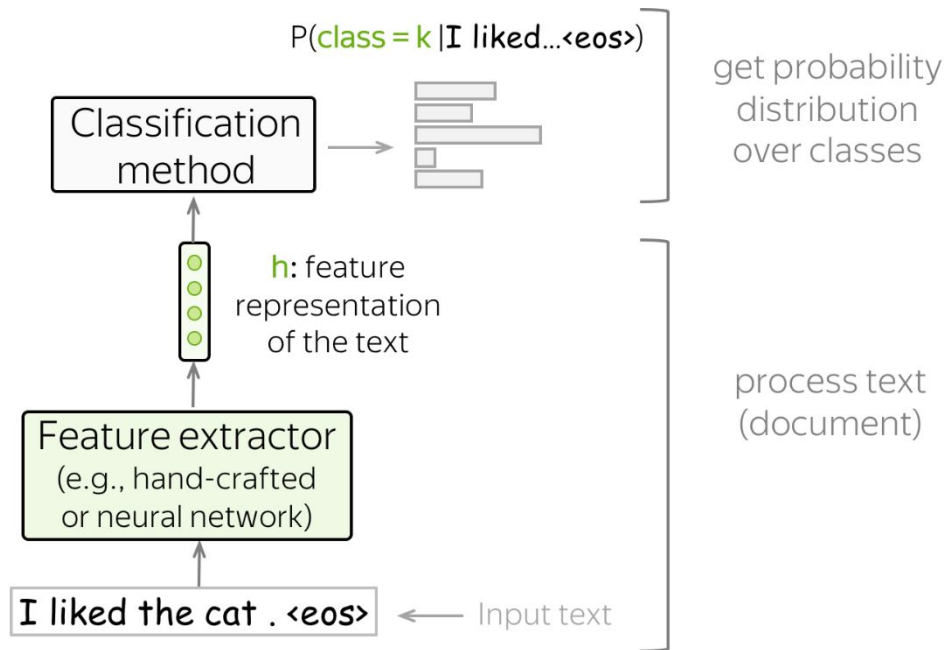
- Примеры
- Последний слой
- Свёрточные архитектуры
- Recurent NN
- Embeddings

Примеры

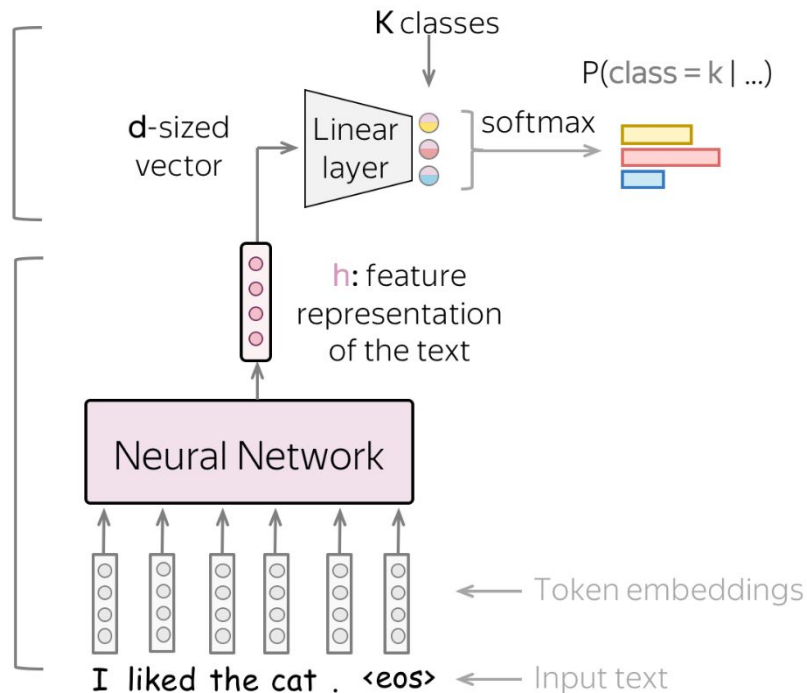


Общая концепция

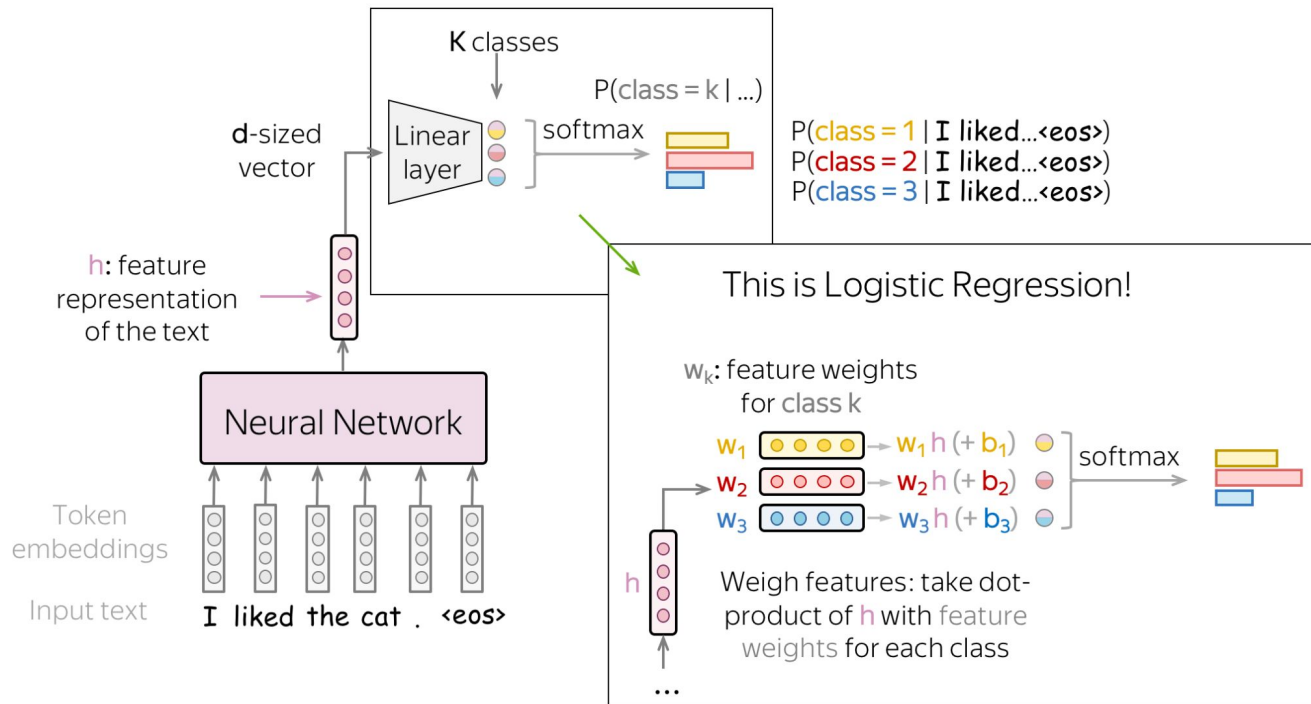
General Classification Pipeline



Classification with Neural Networks



Последний слой



Loss-функция

Training example: **I liked the cat on the mat <eos>**

Label: **k**
↑
target

Model prediction:

$P(\text{class} = i | \text{I liked...<eos>})$



k

Target:

p^*



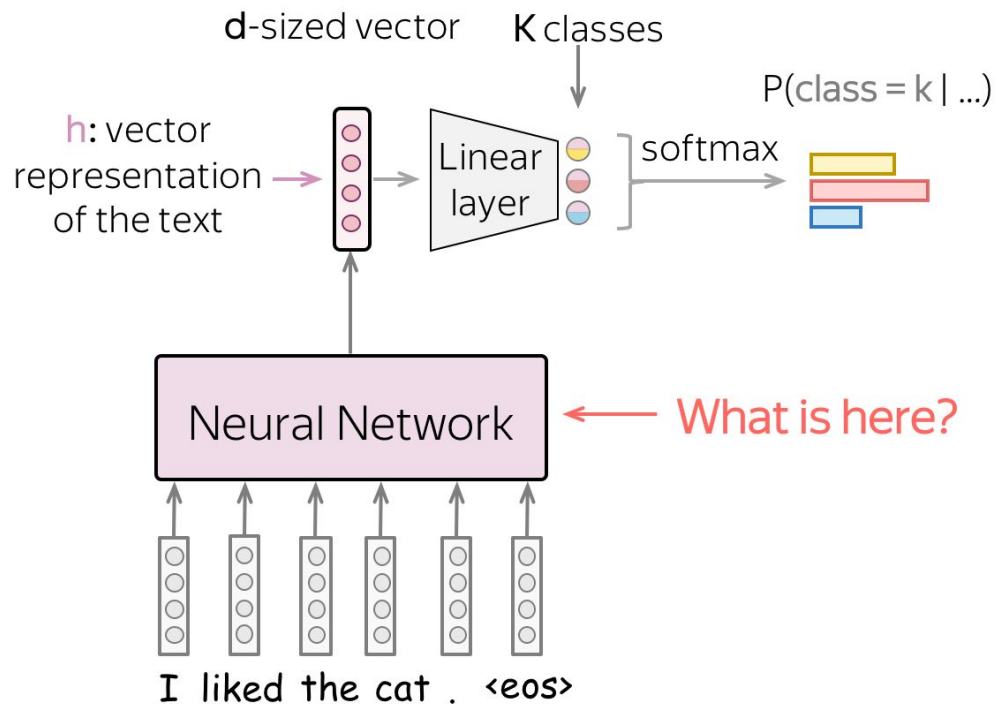
Cross-entropy loss:

$$-\sum_{i=1}^K p_i^* \cdot \log P(y = i|x) \rightarrow \min \quad (p_k^* = 1, p_i^* = 0, i \neq k)$$

For one-hot targets, this is equivalent to

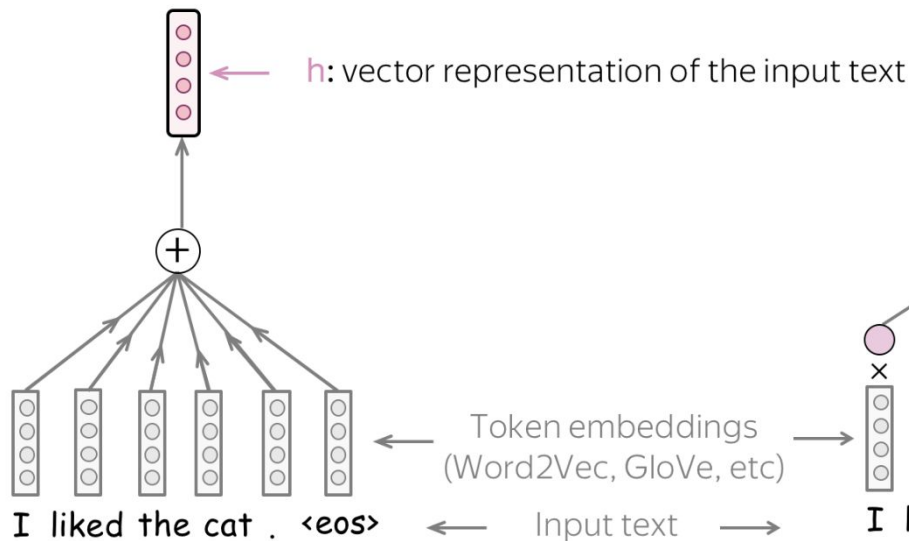
$$-\log P(y = k|x) \rightarrow \min$$

А что внутри?

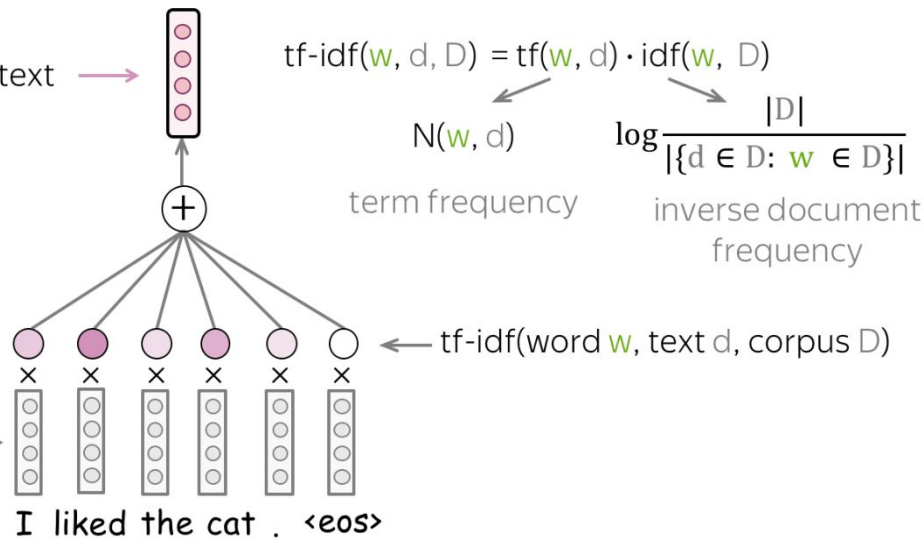


№0. Усреднение эмбедингов

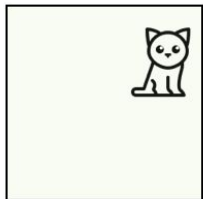
Sum of embeddings
(Bag of Words, Bag of Embeddings)



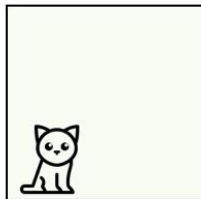
Weighted sum of embeddings
(e.g., using tf-idf weights)



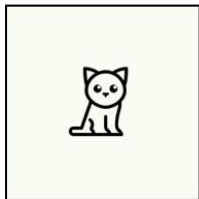
CNN



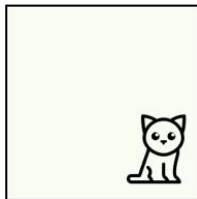
Label: **cat**



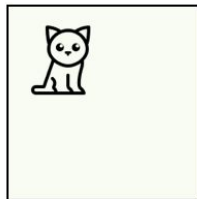
Label: **cat**



Label: **cat**



Label: **cat**



Label: **cat**

We don't care where the cat is,
we care that it is somewhere.

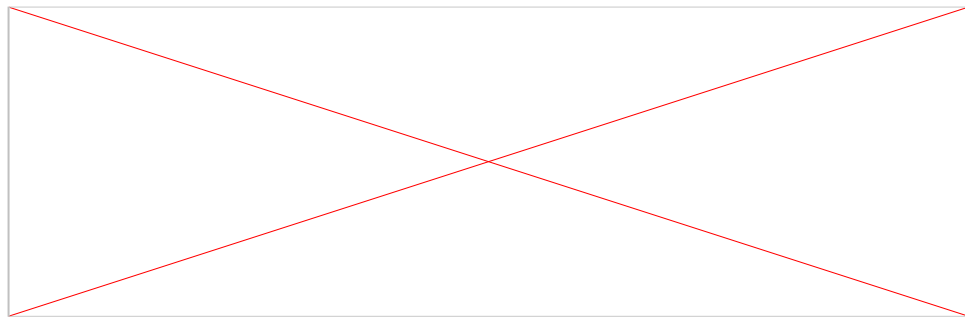
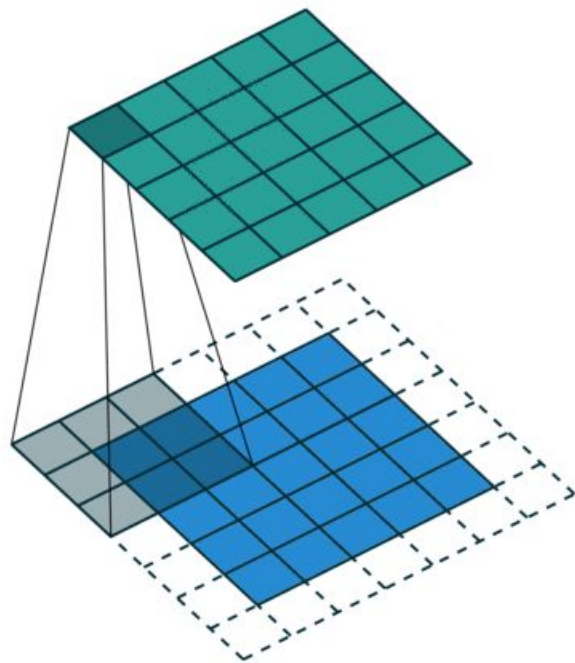
Then why don't we process all
these cats similarly?

An **absolutely great** movie! I watched the premiere with my friends.

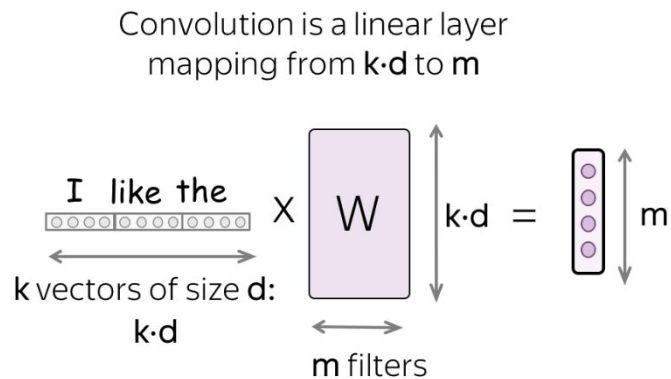
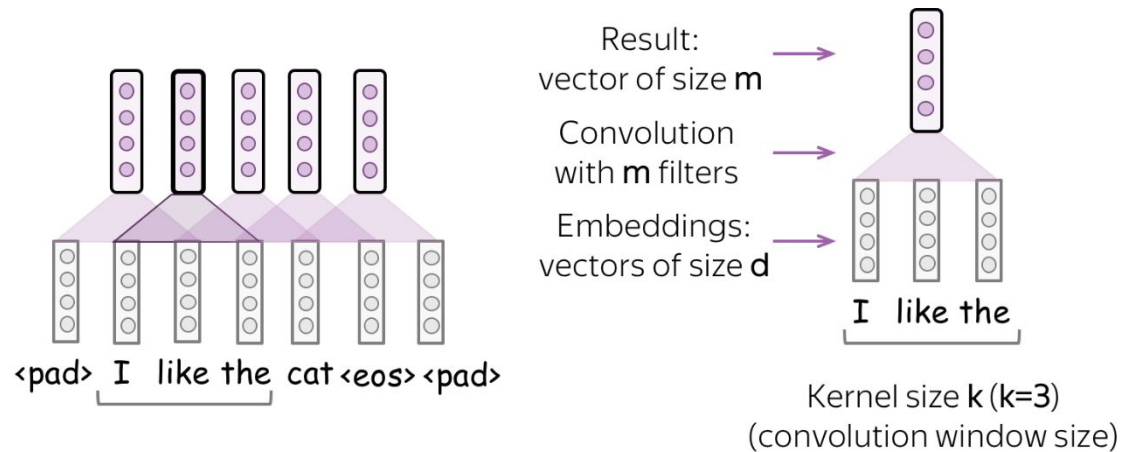
The movie about cats was **absolutely great**, and the cats were cute.

The movie is about cats running around, and it is **absolutely great**.

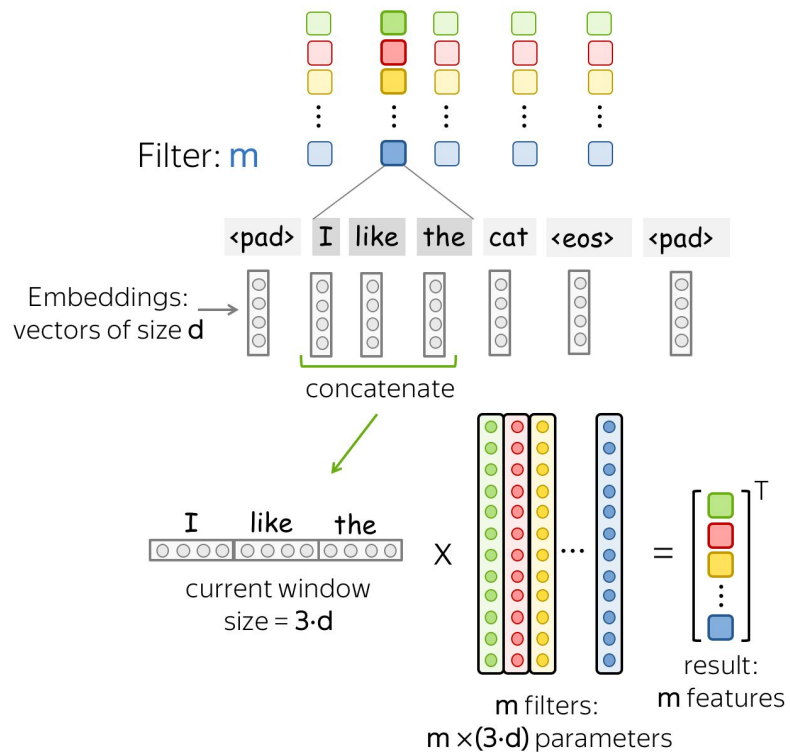
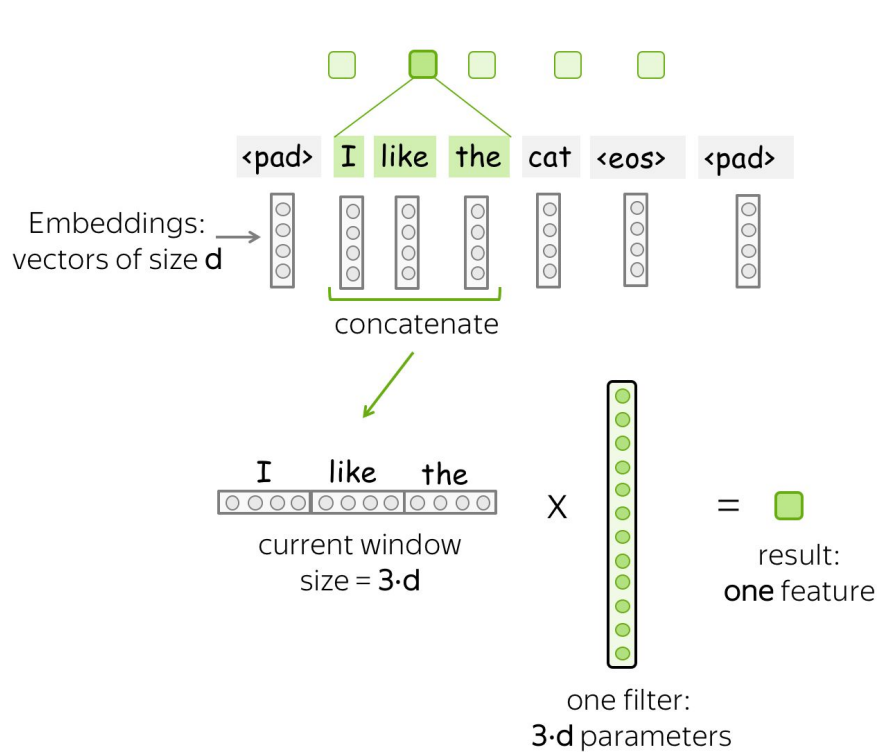
CNN



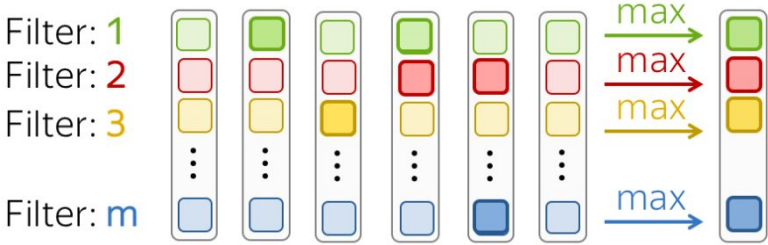
CNN



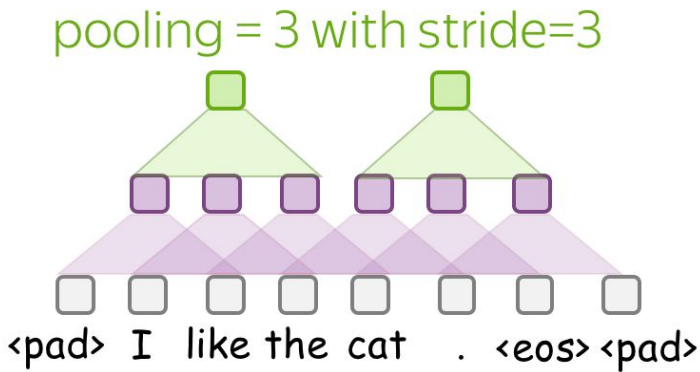
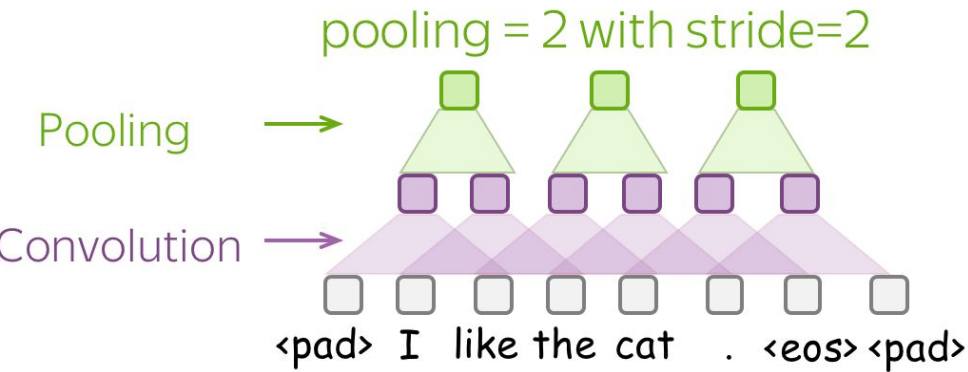
CNN



CNN

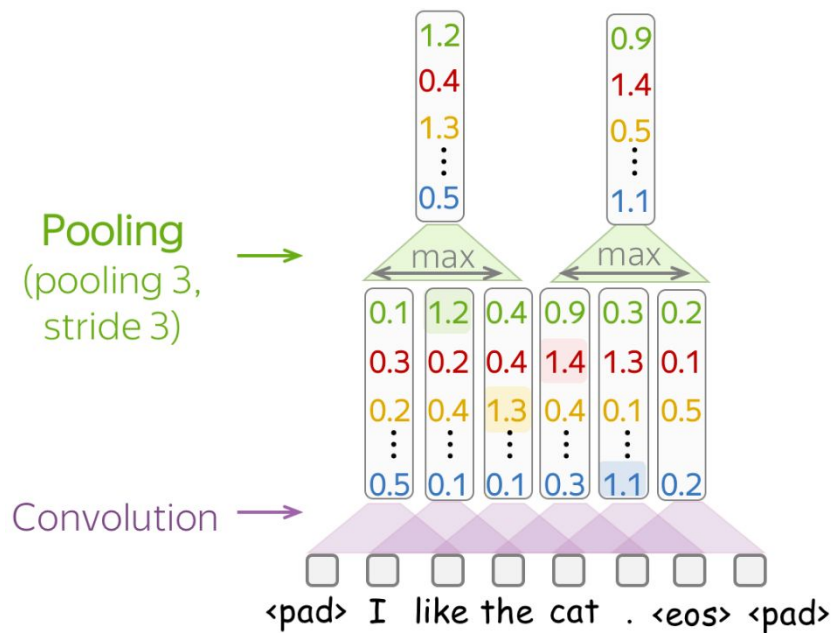


Max pooling:
maximum for each
dimension (feature)

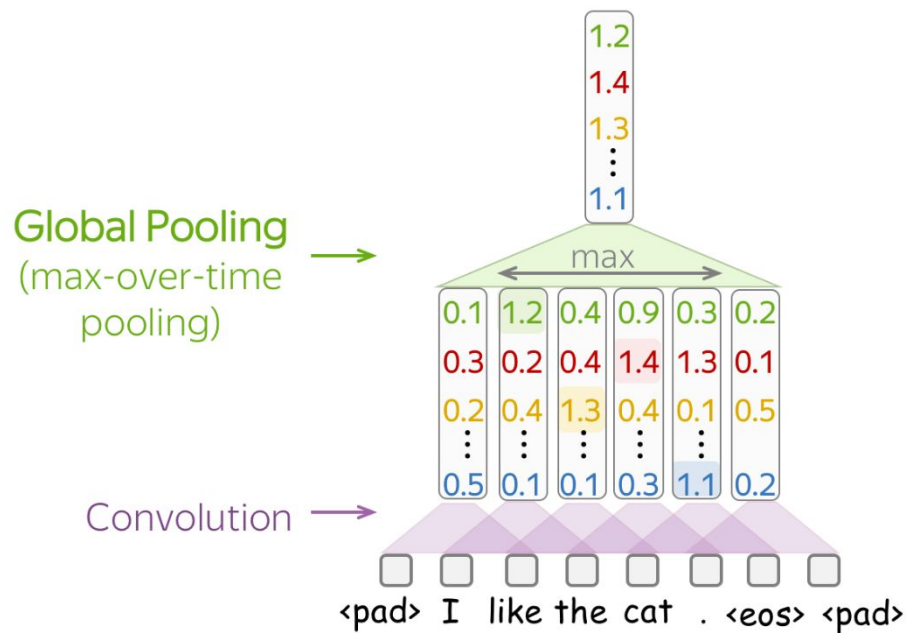


CNN

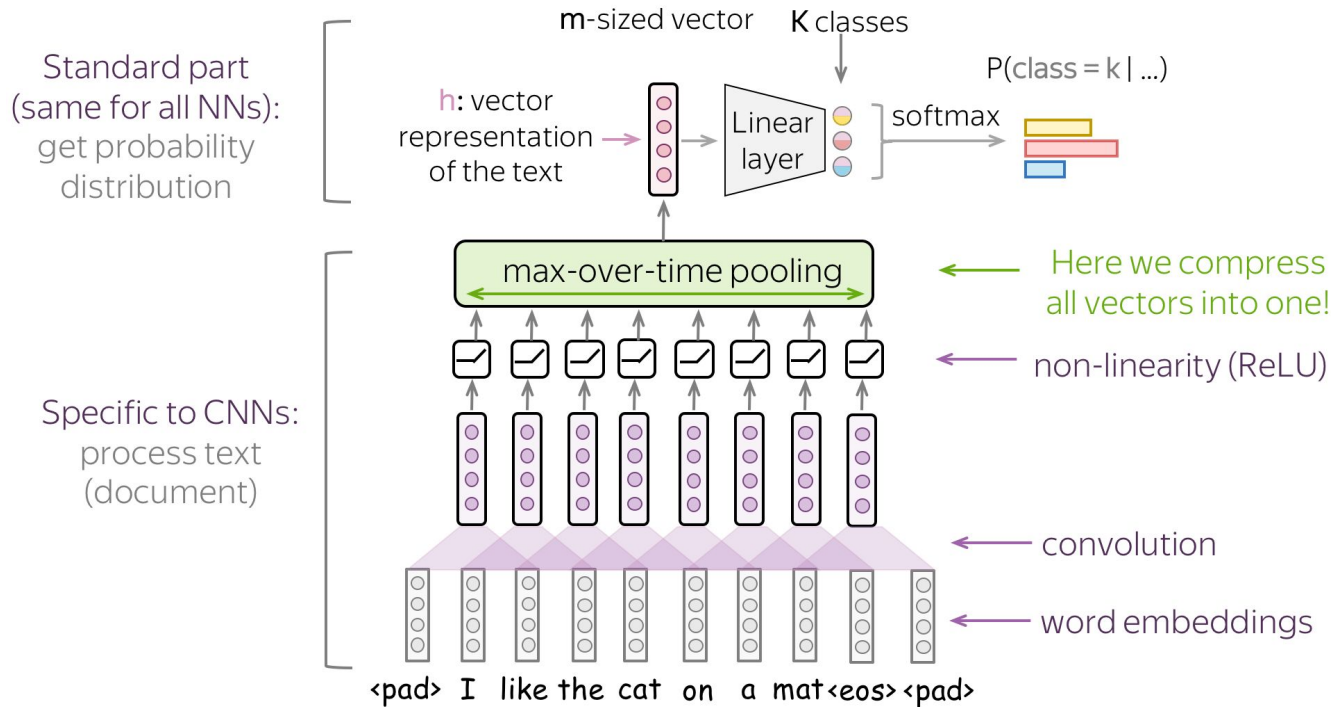
Pooling



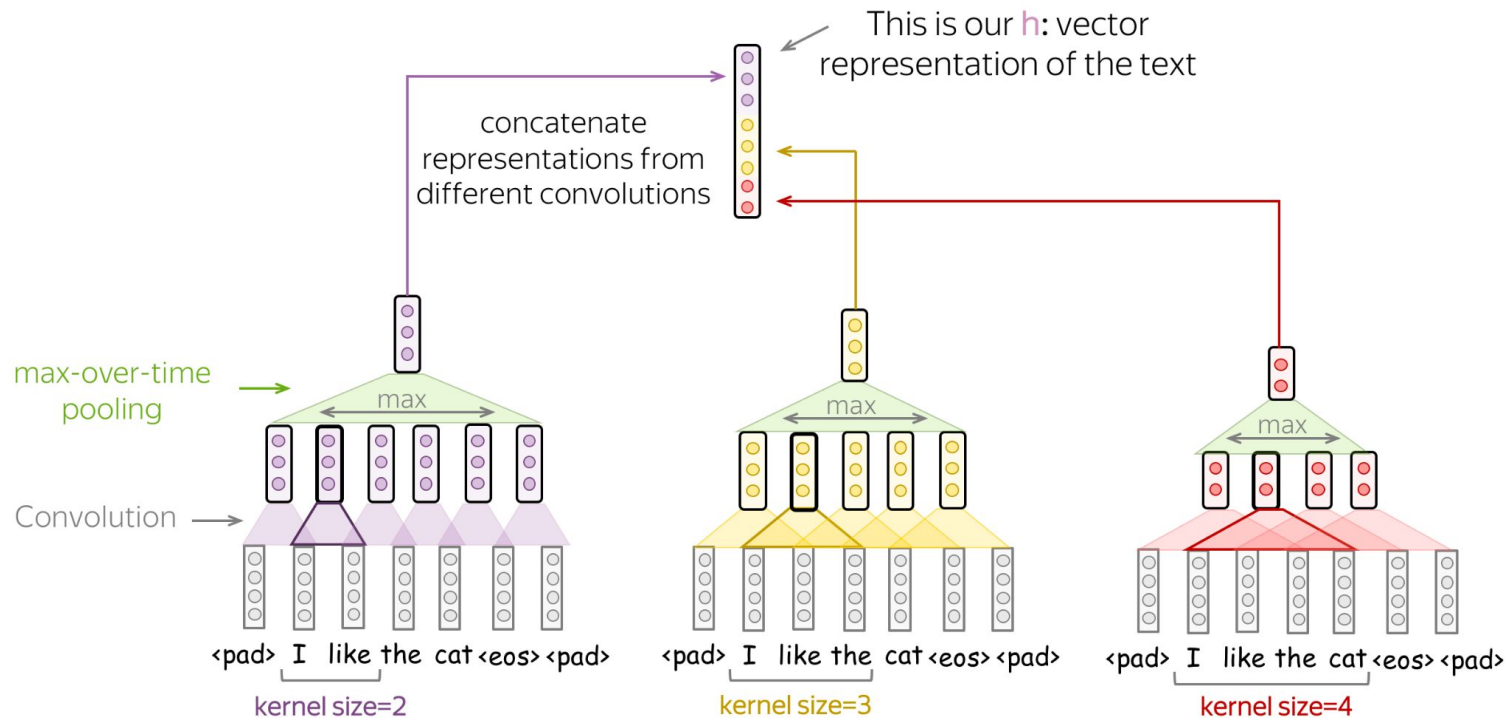
Global Pooling



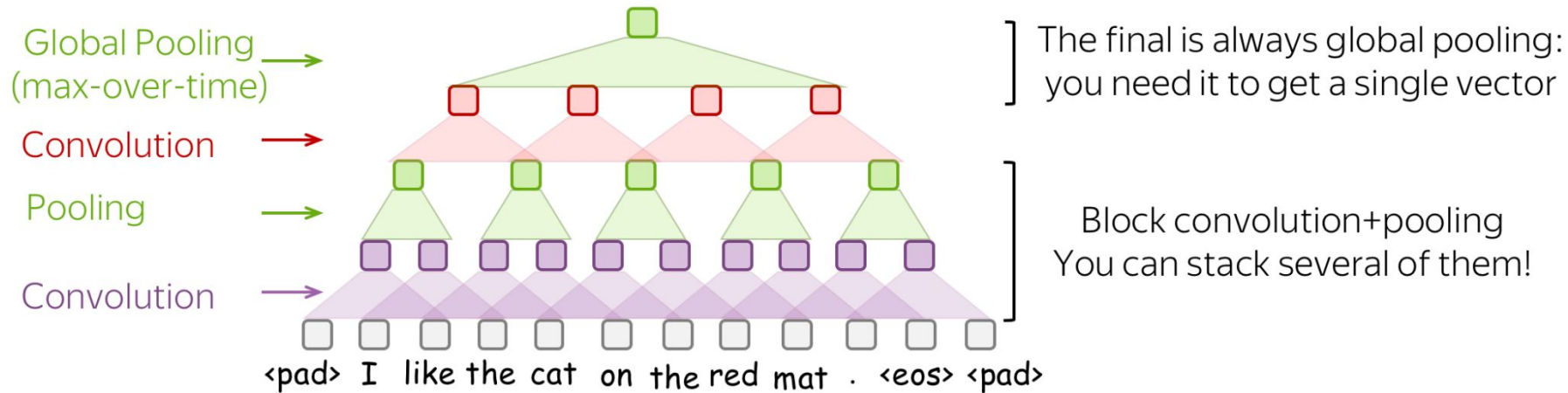
CNN



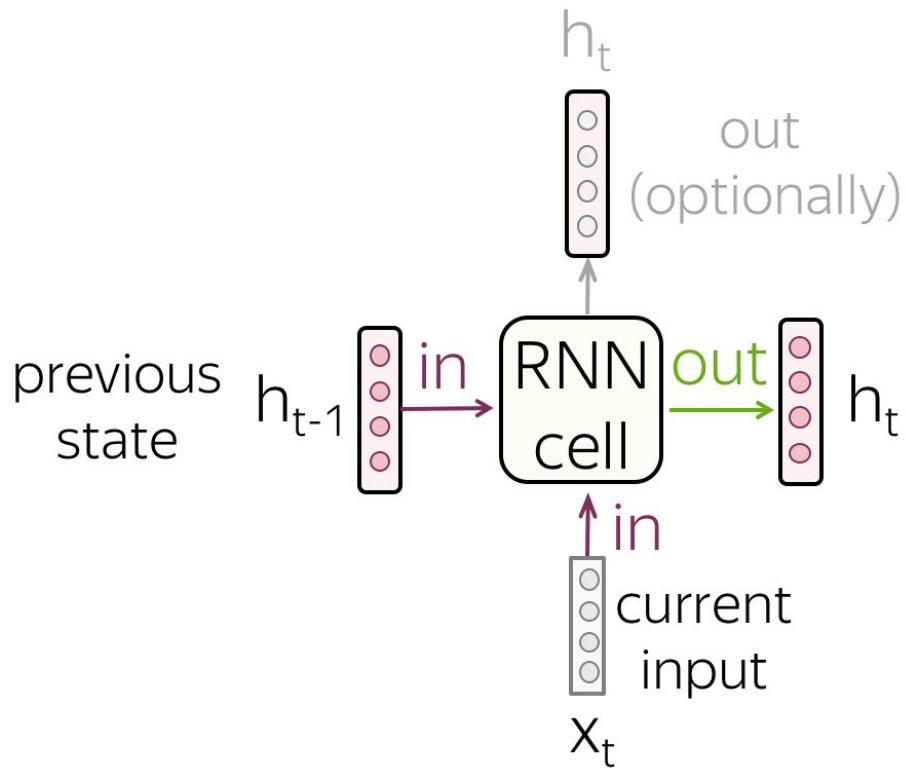
CNN



CNN



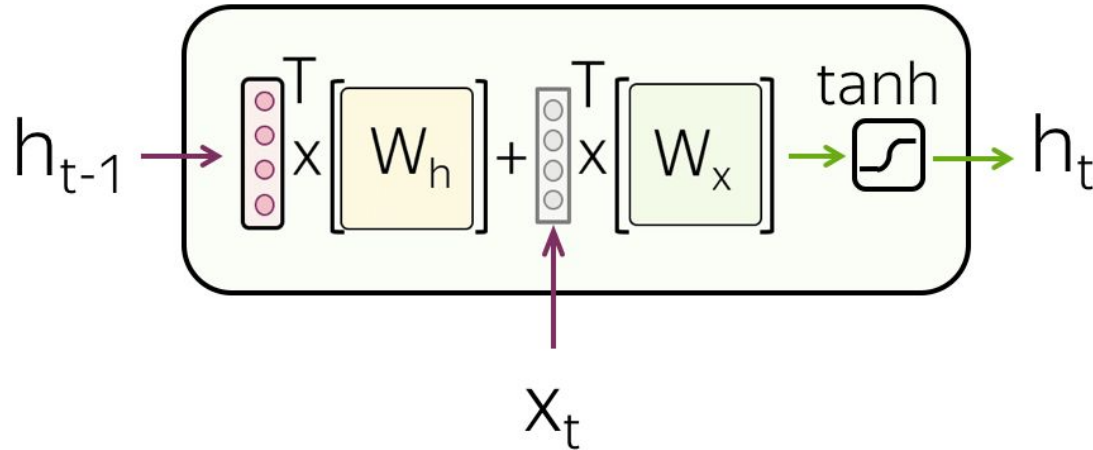
RNN



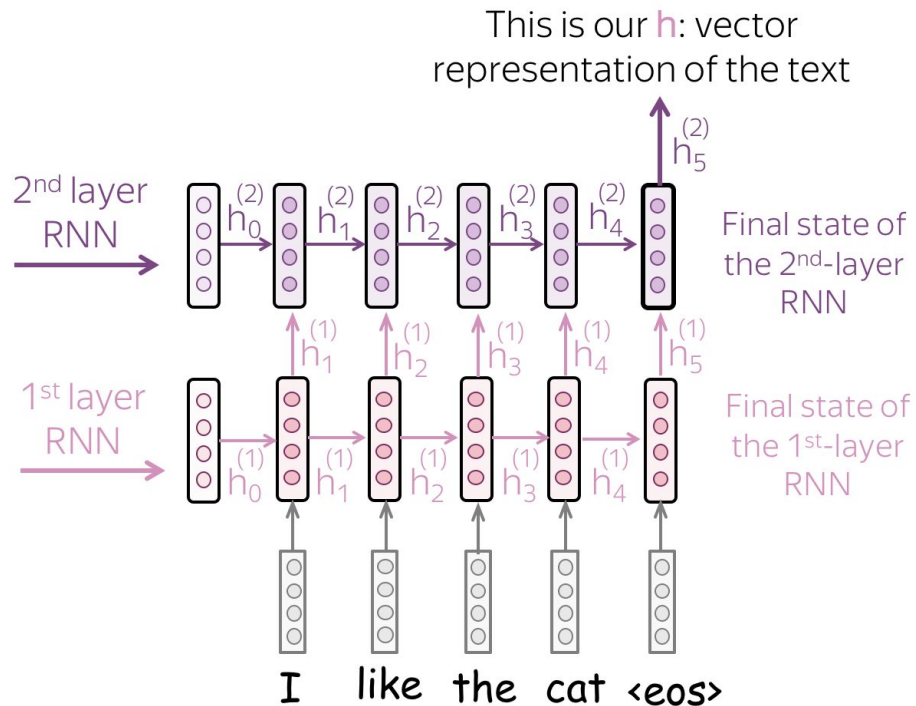
RNN

Vanilla RNN

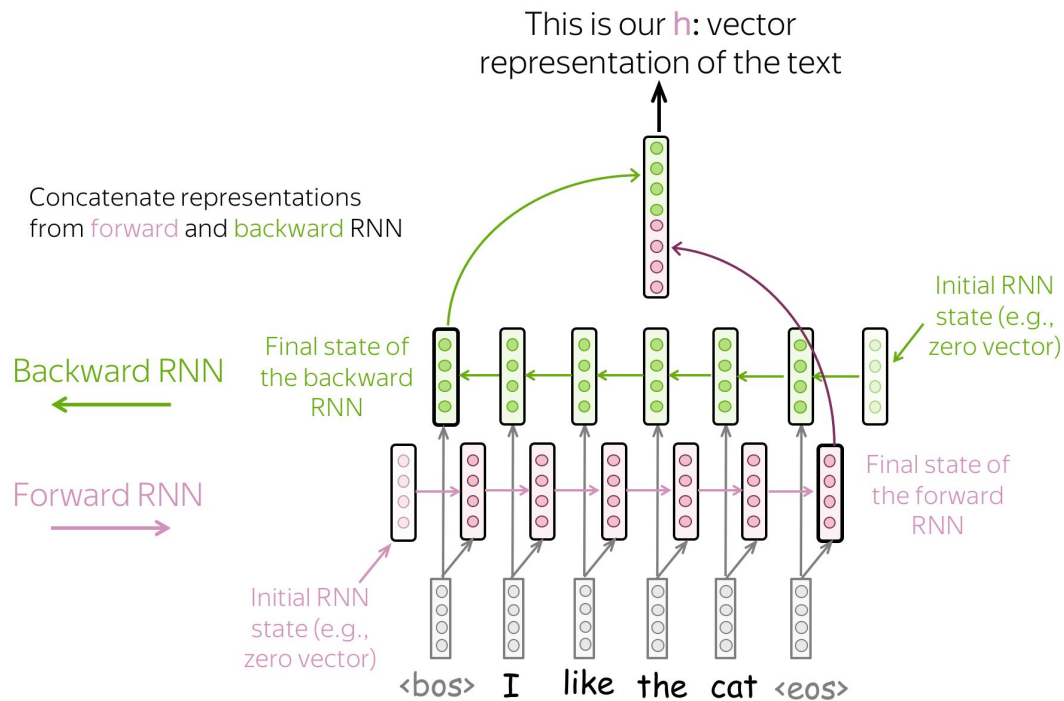
$$h_t = \tanh(h_{t-1}W_h + x_tW_x)$$



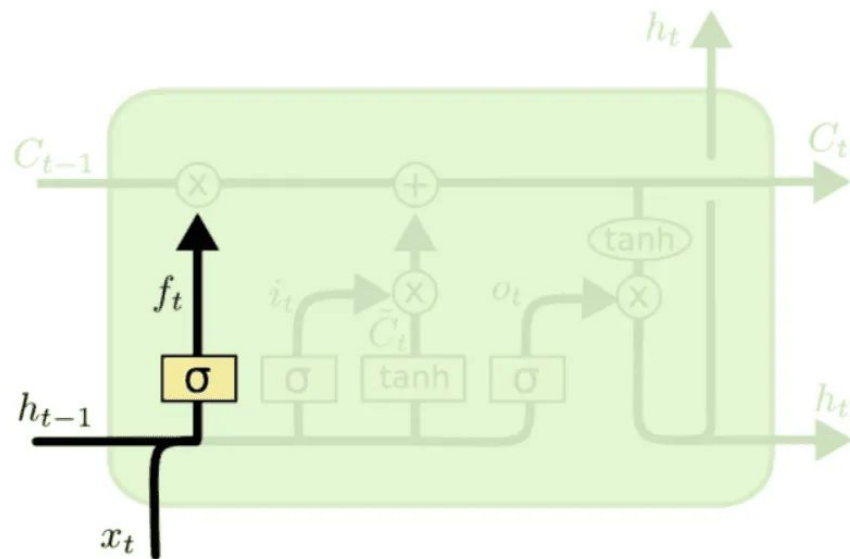
Two layer RNN



Bidirectional RNN

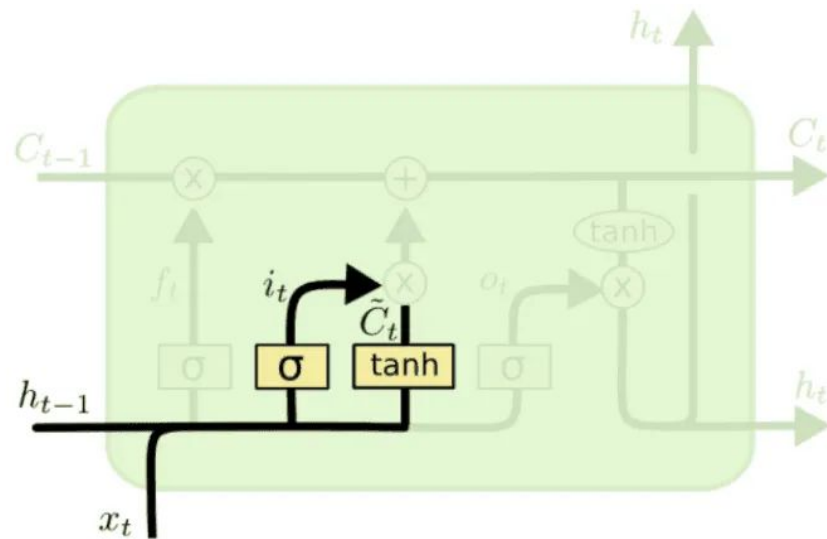


LSTM



$$f_t = \sigma(h_{t-1}W_1^f + x_tW_2^f + b_f)$$

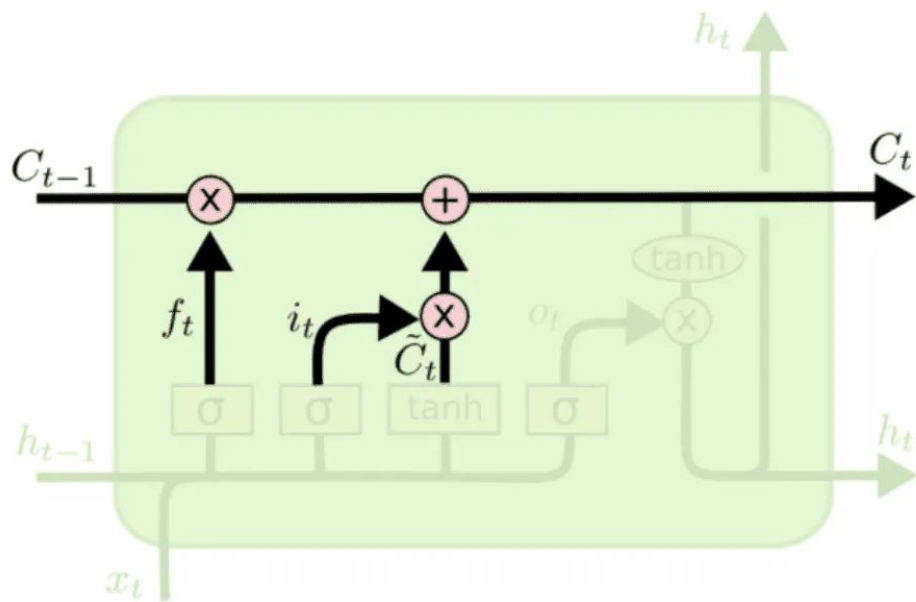
LSTM



$$\tilde{C}_t = \tanh(h_{t-1}W_1^C + x_tW_2^C + b_c)$$

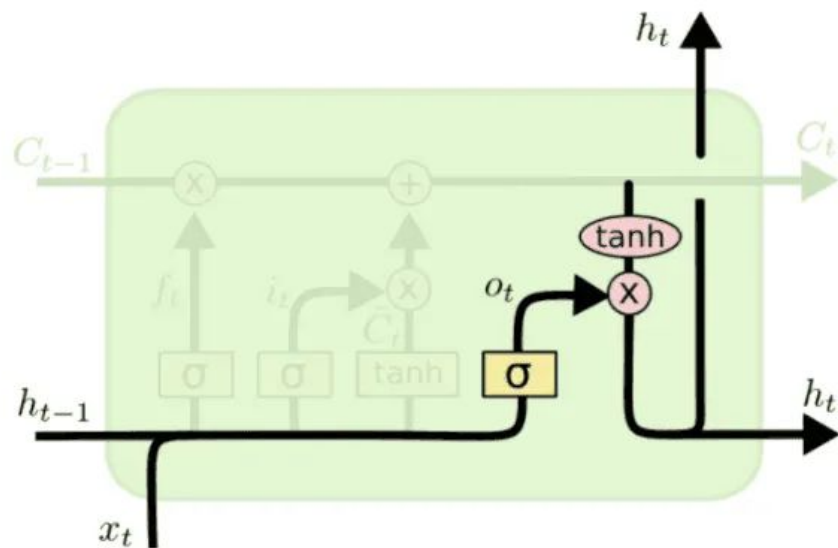
$$i_t = \sigma(h_{t-1}W_1^i + x_tW_2^i + b_i)$$

LSTM



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

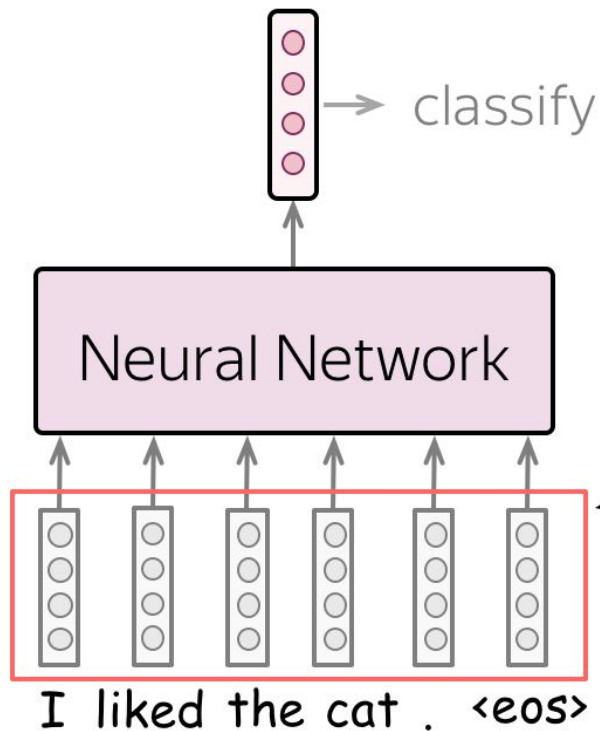
LSTM



$$o_t = \sigma(h_{t-1}W_1^o + x_tW_2^o + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

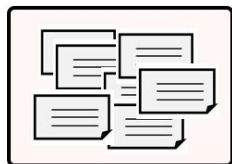
Embeddings



Input word embeddings:

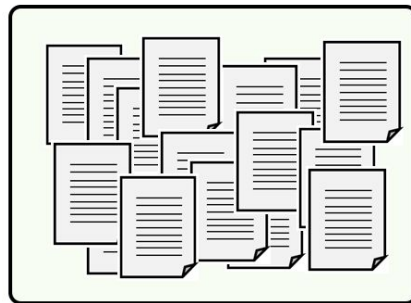
- Train from scratch
- Take pretrained (Word2Vec, GloVe)
- Initialize with pretrained, then fine-tune

Embeddings



Training data for text classification (labeled)

- Not huge, or not diverse, or both
- Domain: task-specific



Training data for word embeddings (unlabeled)

- Huge diverse corpus (e.g., Wikipedia)
- Domain: general

Embeddings

- Train from scratch

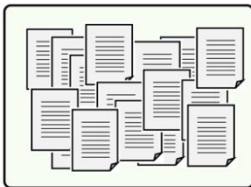
What they will know:



May be not enough to
learn relationships
between words

- Take pretrained (Word2Vec, GloVe)

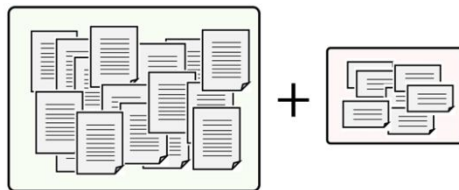
What they will know:



Know relationships between words,
but are **not** specific to the task

- Initialize with pretrained, then fine-tune

What they will know:



Know relationships between
words and adapted for the task

“**Transfer**” knowledge from a huge unlabeled corpus to your task-specific model

The end?

Sequence-to-sequence



Задача

Human Translation

$$y^* = \arg \max_y p(y|x)$$

The “probability” is intuitive and is given by a human translator’s expertise

Machine Translation

$$y' = \arg \max_y p(y|x, \theta)$$

model parameters

Questions we need to answer

- **modeling**

How does the model for $p(y|x, \theta)$ look like?

- **learning**

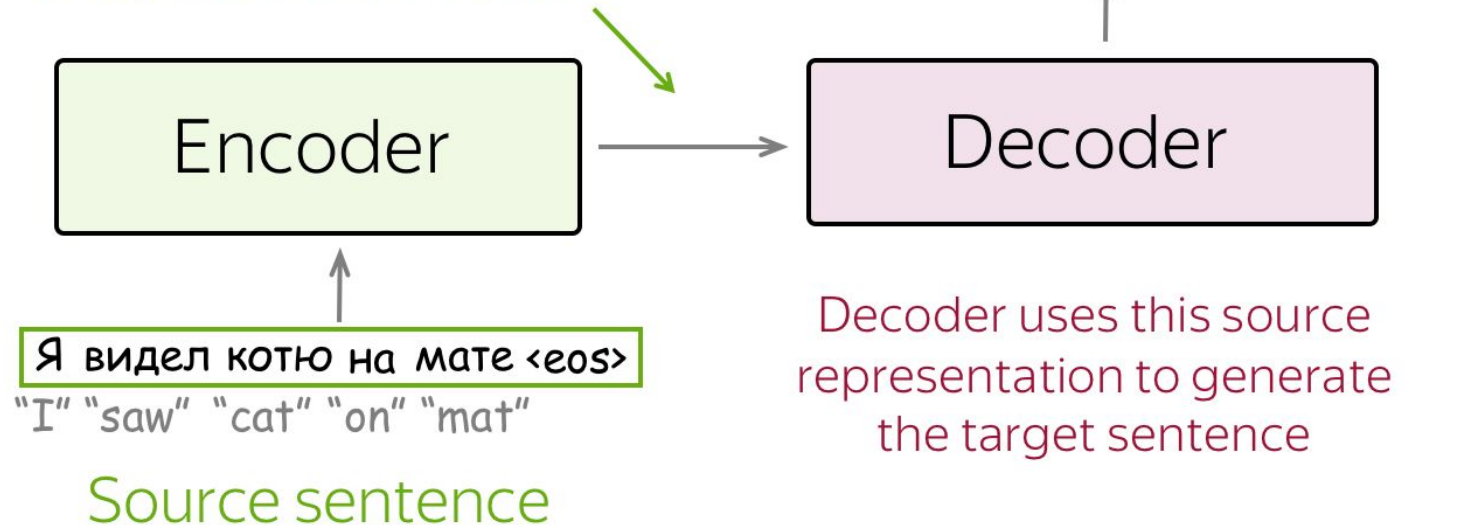
How to find θ ?

- **search**

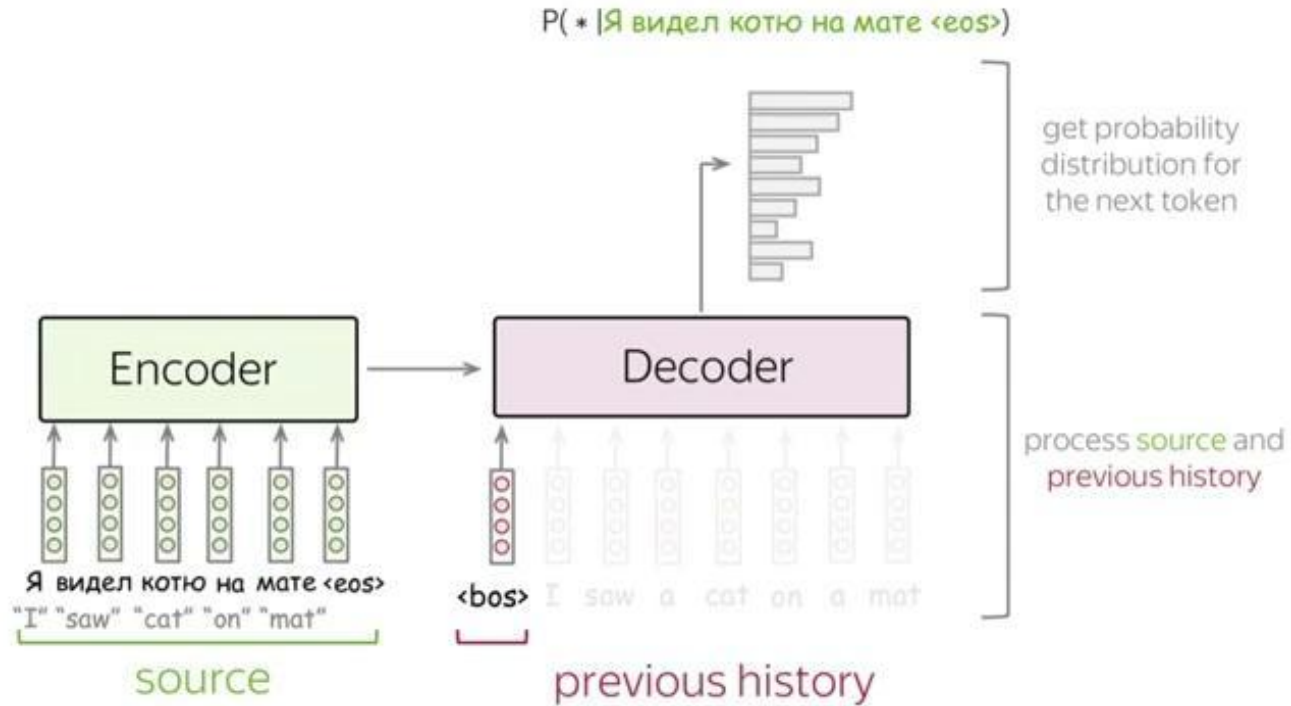
How to find the argmax?

Encoder-decoder

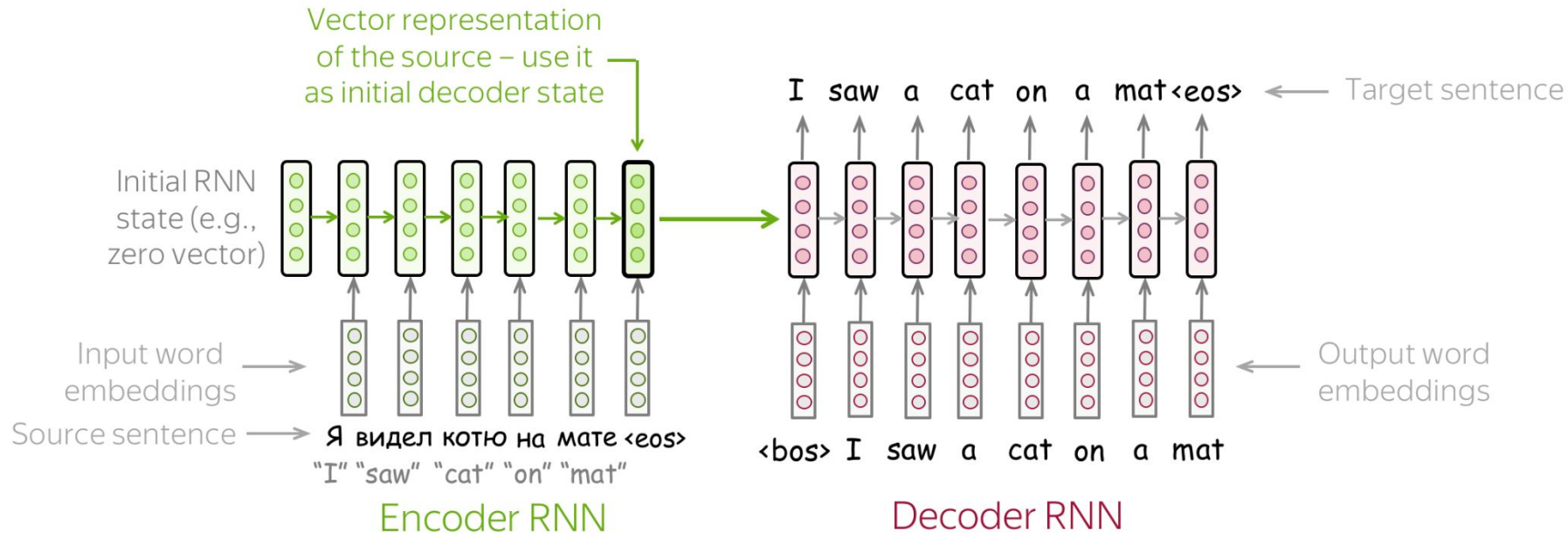
Encoder builds a representation of the source and gives it to the decoder



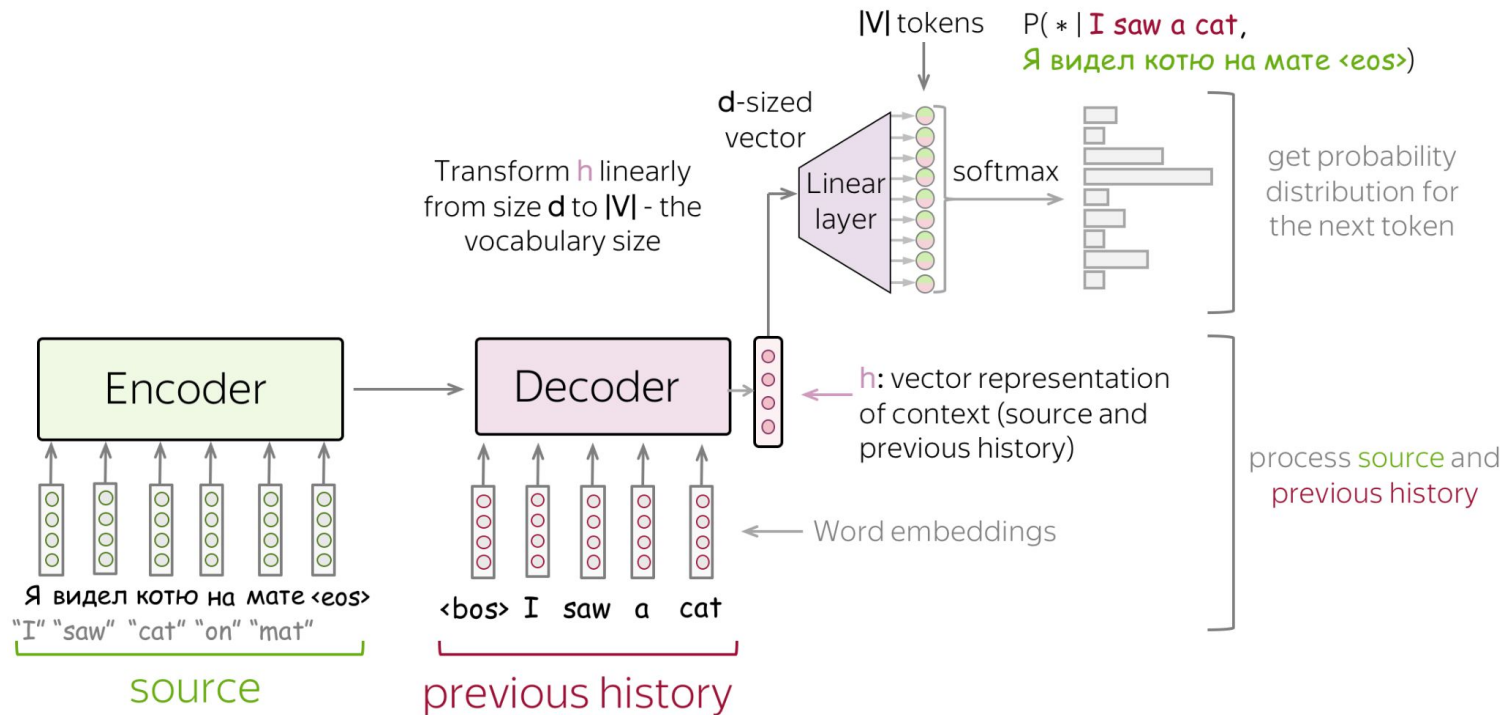
Encoder-decoder



RNN



Как генерируем?



Как генерируем?

Source sequence:

Я видел котю на мате <eos>

"I" "saw" "cat" "on" "mat"

Target sequence:

I saw a cat on a mat <eos>

previous tokens

we want the model
to predict this

← one training example

← one step for this example

Model prediction: $p(* | \text{I saw a, Я ... <eos>})$



← cat →

Target



Loss = $-\log(p(\text{cat})) \rightarrow \min$



decrease

increase

decrease

Как найти argmax?

$$y' = \arg \max_y p(y|x) = \arg \max_y \prod_{t=1}^n p(y_t | y_{<t}, x)$$

How to find the argmax?

Жадный подход

$$\arg \max_y \prod_{t=1}^n p(y_t | y_{<t}, x) \neq \prod_{t=1}^n \arg \max_{y_t} p(y_t | y_{<t}, x)$$

Beam search

<bos>

Start with the begin of sentence token or with an empty sequence

Once again генерация

Encoder: read source



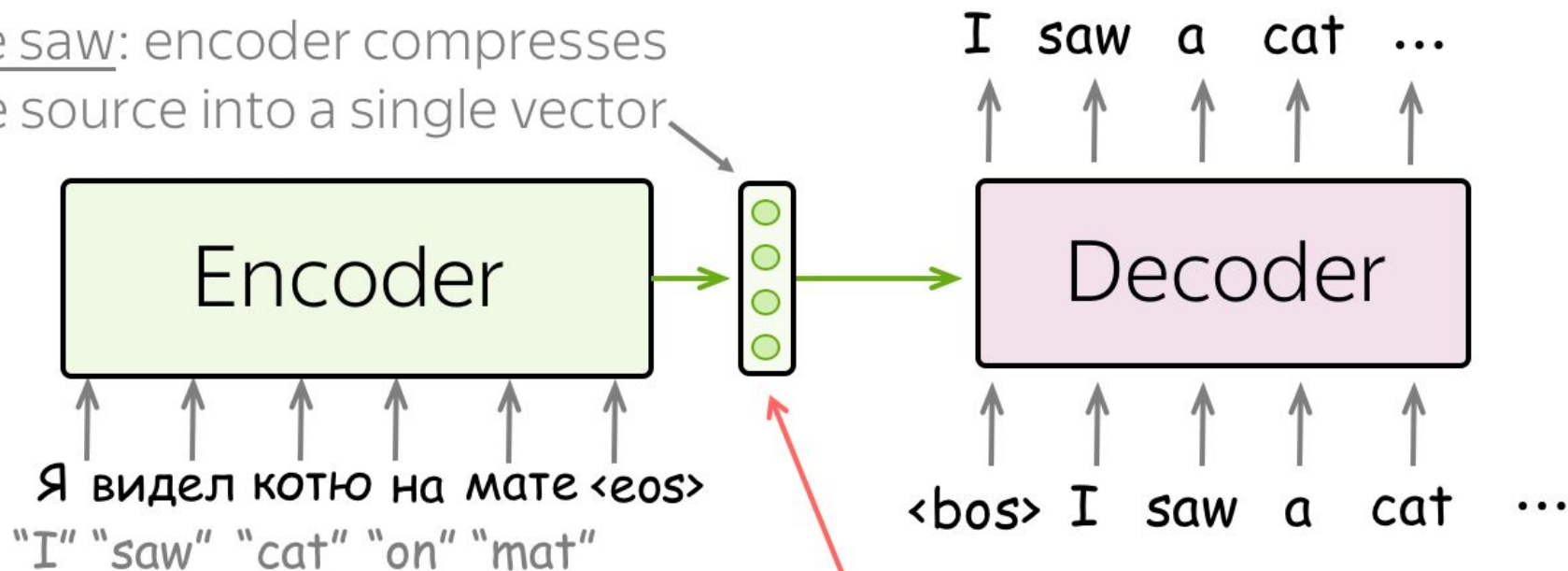
we are here

Source: Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"

Target: I saw a cat on a mat <eos>

Но есть нюанс

We saw: encoder compresses the source into a single vector



Problem: this is a bottleneck!

The end?

Attention

Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens

A model can learn to “pay attention” to the most relevant source tokens for each step

